# Databases for Data Analytics

Task 3 — Joins and DML | Yaron Besser

## Question 1

***Task:*** *All movies with a role 'himself' or 'herself' (case insensitive); multi-role movies counted once.*

**Row count:** 23,525

**SQL Query:**

```
SELECT DISTINCT
    `name`, id, year
FROM
    movies AS m
        JOIN
    roles AS r ON m.id = r.movie_id
WHERE
    LOWER(`role`) IN ('himself', 'herself')
ORDER BY id;
```

**Result:**

| name | id | year |
|---|---|---|
| $5,000,000 Counterfeiting Plot, The | 20 | 1914 |
| $ucces Part One | 32 | 1986 |
| '15' | 36 | 2002 |
| '60s Pop Rock Reunion | 42 | 2004 |
| '70s Celebration: The Beat Is Back, A | 45 | 1993 |
| '70s: The Decade That Changed Television, The | 46 | 2000 |
| '88 Dodge Aries | 47 | 2002 |
| 'Bats' Abound | 64 | 1999 |
| 'Billy Elliot' Boy, The | 67 | 2001 |
| 'Bullitt': Steve McQueen's Commitment to Reality | 72 | 1968 |
| 'Catch Me If You Can': Behind the Camera | 77 | 2003 |

1 19:06:11 SELECT D... 23525 row(s) returned 0.779 sec / 0.0028 sec

**Explanation:**

Joined **movies** with **roles** on movie_id, filtered `LOWER(role)` to 'himself'/'herself' for case-insensitivity, and used **DISTINCT** so a movie with multiple self-roles is counted once.

## Question 2

***Task:*** *All actors that Alfred Hitchcock directed.*

**Row count:** 1,342

**SQL Query:**

```
SELECT DISTINCT
    a.first_name, a.last_name, a.id
FROM
    directors AS d
        JOIN
    movies_directors AS md ON d.id = md.director_id
        JOIN
    roles AS r ON md.movie_id = r.movie_id
        JOIN
    actors AS a ON r.actor_id = a.id
WHERE
    d.first_name = 'Alfred (I)'
        AND d.last_name = 'Hitchcock'
ORDER BY a.id;
```

**Result:**

| first_name | last_name | id |
|---|---|---|
| Alfred | Abel | 801 |
| Walter | Abel | 831 |
| Rodney | Ackland | 1766 |
| Robert | Adair | 2003 |
| Ronald | Adam | 2069 |
| Samuel (I) | Adams | 2474 |
| Stanley (I) | Adams | 2486 |
| George | Adrian | 3094 |
| Brian | Aherne | 3974 |
| Patrick | Aherne | 3978 |
| Charles | Aidman | 4315 |

2 19:06:12 SELECT D... 1342 row(s) returned 0.0067 sec / 0.00008...

**Explanation:**

Chained four joins (**directors** → **movies_directors** → **roles** → **actors**) to link Hitchcock to the actors who appeared in his movies. Filtered on his exact name `'Alfred (I) Hitchcock'` and used **DISTINCT** since an actor may appear in several of his films.

## Question 3

***Task:*** *All genres in which Hitchcock did not direct any movie. Improve lookup performance by creating a dedicated genres table.*

**Row count:** 7 (out of 21 total genres in genres_type)

**SQL Query:**

```
-- Creating a dedicated table for genres
DROP TABLE IF EXISTS genres_type;
CREATE TABLE genres_type AS
    SELECT DISTINCT genre FROM movies_genres;

-- All genres in which Hitchcock did not direct any movie
SELECT gt.genre
FROM genres_type AS gt
    LEFT JOIN (
        SELECT mg.genre AS hitc_genre, d.id
        FROM directors AS d
            JOIN movies_directors AS md ON d.id  = md.director_id
            JOIN movies_genres    AS mg ON md.movie_id = mg.movie_id
        WHERE d.first_name = 'Alfred (I)'
          AND d.last_name  = 'Hitchcock'
    ) AS hitc ON gt.genre = hitc.hitc_genre
WHERE hitc.id IS NULL;
```

**Result:**

| genre |
|---|
| Western |
| Family |
| Animation |
| Adult |
| Action |
| Fantasy |
| Sci-Fi |

| | | | | |
|---|---|---|---|---|
| 3 | 19:06:12 | DROP TAB... | 0 row(s) affected | 0.0018 sec |
| 4 | 19:06:12 | CREATE T... | 21 row(s) affected Records: 21 Duplicates: 0 Warnings: 0 | 0.095 sec |
| 5 | 19:06:12 | SELECT... | 7 row(s) returned | 0.0015 sec / 0.00000... |

**Explanation:**

Built a dedicated **genres_type** table holding each genre once (21 rows). Used a **LEFT JOIN** against a subquery containing every genre Hitchcock directed in; rows with no match get `NULL` on the right side, so `WHERE hitc.id IS NULL` returns exactly the 7 genres he never worked in — an anti-join, equivalent to `NOT IN`. **Why pull d.id into the subquery:** `d.id` is the primary key of **directors**, so it is guaranteed `NOT NULL` on every matched row. That makes `hitc.id IS NULL` a reliable anti-join signal — a `NULL` there can only mean 'no match from the LEFT JOIN', never a genuine missing value. Checking a nullable column instead would risk false positives.

## Question 4

***Task:*** *Copy movies to sad_movies and delete all that belong to the genre 'Comedy'.*

**Row count:** 331,844 remaining in sad_movies (388,269 – 56,425 deleted)

**SQL Query:**

```
-- Copy movies into sad_movies
DROP TABLE IF EXISTS sad_movies;
CREATE TABLE sad_movies AS
    SELECT id, `name` FROM movies;

-- Delete all that belong to the genre 'Comedy'
DELETE sm FROM sad_movies AS sm
    JOIN (
        SELECT id
        FROM movies AS m
            JOIN movies_genres AS mg ON m.id = mg.movie_id
        WHERE mg.genre = 'Comedy'
    ) AS com_movies ON sm.id = com_movies.id;

-- Testing: should return 0 rows (no Comedy left in sad_movies)
SELECT *
FROM sad_movies AS sm
    JOIN movies_genres AS mg ON mg.movie_id = sm.id
WHERE genre = 'Comedy';
```

**Result:**

| id | name | movie_id | genre |
|---|---|---|---|

| | | | | |
|---|---|---|---|---|
| 6 | 19:06:12 | DROP TAB... | 0 row(s) affected | 0.0027 sec |
| 7 | 19:06:12 | CREATE T... | 388269 row(s) affected Records: 388269 Duplicates: 0 Warnings: 0 | 0.621 sec |
| 8 | 19:06:13 | DELETE s... | 56425 row(s) affected | 0.587 sec |
| 9 | 19:06:13 | SELECT... | 0 row(s) returned | 0.250 sec / 0.000038... |

**Explanation:**

Copied **movies** into **sad_movies** with `CREATE TABLE AS SELECT`, then used **DELETE ... JOIN** against a subquery of Comedy movie ids — the inner join keeps only Comedy rows and DELETE removes them. Result: 56,425 Comedy movies deleted, 331,844 non-Comedy remain (388,269 – 56,425). The test query returns 0 rows, confirming no Comedy survived.